

计算机图形学 期中考复习指南

Computer Graphics Midterm Review Guide

参考教材：《计算机图形学基础教程》（清华大学出版社）

姓名 (Name): 尤晟喧 | 学号 (ID): R32314015

专业 (Major): 数字媒体技术 (Digital Media Technology)

院校 (Univ): 安徽大学 (Anhui University)

第一章：绪论 (Introduction to CG)

- 图形学之父 (Father of Computer Graphics):** Ivan Sutherland。他在1963年开发了 **Sketchpad** 系统，提出了人机交互和面向对象设计的核心概念。
- 重要会议 (Important Conferences):**
 - SIGGRAPH (Special Interest Group on Computer Graphics and Interactive Techniques):** 国际最权威的图形学顶会。
 - Eurographics / Pacific Graphics** 等。
- 重要奖项 (Important Awards):**
 - 图灵奖 (ACM A.M. Turing Award):** 计算机界最高奖项。多位图形学巨匠曾获此殊荣（如 Ivan Sutherland，以及皮克斯动画的 Ed Catmull 和 Pat Hanrahan）。
- 相关设备 (Hardware Devices):**
 - 显示设备 (Display Devices):** 阴极射线管 (CRT)、液晶显示器 (LCD)、图形处理器 (GPU)。
 - 交互与输入输出设备 (I/O Devices):** 数字化仪、光笔、绘图仪 (Plotter) 以及 **虚拟现实头显 (VR Headset, 如 Meta Quest 3S)** 等先进交互终端。

第二章：图形学基础 (Basics of CG)

*注：根据考试要求，本部分不考查 *OpenGL*，重点掌握变换及其关系与颜色模型。

1. 各种变换及其关系 (Transformations and Their Relationships)

几何变换是对图形在空间中进行操作的数学基础。它们存在明确的包含关系：

- 刚体变换 (Rigid-body Transformation):** 仅包含平移 (Translation) 和旋转 (Rotation)。保持图形的长度、角度和面积不变（即等距变换）。

- **仿射变换 (Affine Transformation):** 包含刚体变换，加上缩放 (Scaling) 和错切 (Shearing)。
重要特性 (Key Property): 保持平行线依旧平行，但长度和角度可能改变。
- **投影变换 (Projective Transformation):** 包含仿射变换。用于将 3D 降维到 2D。
重要特性: 保持直线依旧是直线 (Preserves colinearity)，但不保证平行线的平行性。
- **关系树 (Hierarchy):** 刚体变换 $\subset$ 仿射变换 $\subset$ 投影变换。

2. 四种颜色模型 (Four Color Models)

- **RGB 模型 (Red, Green, Blue): 加色模型 (Additive Color Model)。** 主要用于发光设备（如显示器、手机屏幕）。三原色相加得到白色。
- **CMYK 模型 (Cyan, Magenta, Yellow, Black): 减色模型 (Subtractive Color Model)。** 主要用于打印和印刷 (Printing)。颜色叠加吸收光线，最终得到黑色。
- **HSV/HSB 模型 (Hue, Saturation, Value/Brightness):** 基于人类视觉感知的模型。色调 (Hue, 颜色种类)、饱和度 (Saturation, 色彩纯度)、明度 (Value, 亮度)。
- **CIE 色度模型 (CIE Chromaticity Model):** 基于客观物理量测量的国际标准模型（如 CIE-XYZ），是定义其他颜色模型的数学基准。

第三章：变换与观察 (Transformation and Viewing)

1. 几何变换 (Geometric Transformations)

使用 **齐次坐标 (Homogeneous Coordinates)** 来将所有变换（包括平移）统一表示为矩阵乘法 (Matrix Multiplication)。二维坐标 (x, y) 表示为 $(x, y, 1)$ 。

- **平移 (Translation):** 点的位置沿着向量移动。
- **缩放 (Scaling):** 相对于原点，坐标乘以比例因子。
- **旋转 (Rotation):** 绕原点逆时针旋转 θ 角度。

2. 透视与投影变换 (Perspective and Projection Transformations)

将 3D 坐标映射到 2D 投影平面上。

- **平行投影 (Parallel Projection):** 投影线相互平行。分为正交投影 (Orthographic) 和斜投影 (Oblique)。不产生近大远小的效果。
- **透视投影 (Perspective Projection):** 投影线相交于 **投影中心 (Center of Projection, COP)**。产生“近大远小”的真实感视觉效果。
 - 根据主灭点 (Vanishing Points) 的数量，分为 **一点透视 (1-point)**、**两点透视 (2-point)** 和 **三点透视 (3-point)**。

3. 二维观察的流程和效果 (2D Viewing Pipeline and Effects)

- **窗口 (Window)**: 世界坐标系中要显示的区域（决定“看什么”）。
- **视区 (Viewport)**: 设备坐标系中用于显示的屏幕区域（决定“显示在何处”）。
- **流程 (Pipeline)**: 世界坐标系 (WC) → 观察坐标系 (VC) → 归一化设备坐标系 (NDC) → 设备坐标系 (DC)。
- **效果 (Effects)**: 如果窗口与视区的纵横比 (Aspect Ratio) 不一致，图形会发生拉伸变形 (Distortion)。

第四章：光栅化与扫描转换 (Rasterization and Scan Conversion)

光栅化 (Rasterization): 将几何图形的数学描述转换为像素点矩阵的过程。

1. 直线段的扫描转换 (Line Scan Conversion)

- **DDA 算法 (Digital Differential Analyzer)**: 基于微分的直观算法，计算 $y_{i+1} = y_i + k$ 。缺点是包含浮点运算和四舍五入，效率低。
- **Bresenham 算法**: 图形学中最经典的画线算法。核心思想是引入 **决策变量 (Decision Parameter, d)**，每次迭代仅使用 **整数加减法和位移运算 (Integer arithmetic)** 决定下一个像素位置，效率极高。
- **中点画线法 (Midpoint Line Algorithm)**: 取两个候选像素的中点，代入直线隐式方程 $F(x,y)=ax+by+c$ 进行判定。

2. 圆的扫描转换 (Circle Scan Conversion)

- **八分对称性 (8-way Symmetry)**: 圆具有高度对称性，只需计算 $1/8$ 圆弧（通常是 0 到 45° ），即可通过对称性映射出其余全部像素。
- **中点画圆法 (Midpoint Circle Algorithm)**: 利用圆的隐式方程构造决策变量进行整数迭代。

3. 多边形的扫描转换 (Polygon Scan Conversion)

- **扫描线算法 (Scan-line Algorithm)**: 利用扫描线从下往上扫描多边形，求交点并填充交点间的区间。
- **核心数据结构**: 依赖于 **边表 (Edge Table, ET)** 保存所有的非水平边，以及 **活跃边表 (Active Edge Table, AET)** 动态保存当前扫描线相交的边。

4. 区域填充算法 (Region Filling Algorithms)

- **种子填充算法 (Seed Fill / Flood Fill)**: 给定多边形内部的一个种子像素 (Seed Pixel), 压入栈 (Stack) 中, 然后向相邻像素扩展并填充, 直到遇到边界。
- **连通性 (Connectivity)**: 分为 **4连通 (4-connected)** 和 **8连通 (8-connected)**。

第五章：裁剪 (Clipping) --- 重点考点

Cohen-Sutherland 算法 (区域编码法) - 【必考计算题】

核心思想: 将屏幕划分为9个区域, 赋予 **4位区域编码 (4-bit Outcode)**: TBRL (上 下 右 左)。

- **上 (Top)**: 位4 = 1 ($y > y_{max}$)
- **下 (Bottom)**: 位3 = 1 ($y < y_{min}$)
- **右 (Right)**: 位2 = 1 ($x > x_{max}$)
- **左 (Left)**: 位1 = 1 ($x < x_{min}$)
- **窗口内 (Inside window)** 的编码始终为: 0000

判定逻辑与计算流程: 假设线段端点编码为 C_1 和 C_2

1. **简取 (Trivial Accept)**: 若 $C_1 | C_2 == 0000$ (按位或运算为0, 两端都在窗口内), 则完全保留。
2. **简弃 (Trivial Reject)**: 若 $C_1 \& C_2 \neq 0000$ (按位与运算非0, 同在某一外侧), 则完全舍弃。
3. **求交 (Intersection)**: 否则线段必与边界相交。按照固定顺序计算交点, 将线段分段并舍弃外部部分, 更新新端点的编码, 循环判定。

- **中点分割算法 (Midpoint Subdivision Algorithm)**:

同样使用区域编码进行判定。不直接使用直线方程算交点, 而是通过 **二分法 (Binary Search)** 不断求线段中点来逼近交点。适合硬件实现。

Liang-Barsky 算法 (参数化裁剪) - 【核心算法重点】

核心思想: 利用直线的 **参数方程 (Parametric Equation)**, 将2D的裁剪问题转化为1D参数空间上的不等式求解。

数学推导 (Mathematical Derivation):

线段参数方程: $x = x_1 + u\Delta x, y = y_1 + u\Delta y$ ($0 \leq u \leq 1$)

裁剪条件转化为 4 个不等式:

$$u \cdot p_k \leq q_k \quad (k = 1, 2, 3, 4)$$

- k=1 (左): $p_1 = -\Delta x, q_1 = x_1 - x_{min}$
- k=2 (右): $p_2 = \Delta x, q_2 = x_{max} - x_1$
- k=3 (下): $p_3 = -\Delta y, q_3 = y_1 - y_{min}$
- k=4 (上): $p_4 = \Delta y, q_4 = y_{max} - y_1$

参数 u 的分析:

- 若 $p_k = 0$ 且 $q_k < 0$: 直接舍弃。
- 若 $p_k < 0$: 线段从外部 **进入 (Entering)** 内部。求入点 u 。
- 若 $p_k > 0$: 线段从内部 **离开 (Exiting)** 外部。求出点 u 。
- **计算最终端点:** 真实起点 $u_1 = \max(0, \text{入点 } u)$, 真实终点 $u_2 = \min(1, \text{出点 } u)$ 。
- 若 $u_1 > u_2$ 则完全裁剪。否则代入参数方程得新坐标。

• Sutherland-Hodgeman 算法 (多边形逐边裁剪):

- **核心思想:** 将多边形看作顶点序列。依次针对窗口的四条边界（左、右、下、上）进行裁剪。
- **流水线过程 (Pipeline):** 前一条边界裁剪出的新顶点序列，直接作为下一条边界裁剪的输入序列。
- **四种情况:** 从内到内（输出端点）、从内到外（输出交点）、从外到外（不输出）、从外到内（输出交点，再输出端点）。

Good Luck on Your Midterm Exam! May your code compile and your graphics render flawlessly.